What is encryption and why do we need it? (symmetric vs. asymmetric, the key exchange problem)

Monday 5/4 Notes

Goal: Understand the core problem that encryption solves, and the difference between symmetric and asymmetric encryption

Key Exchange Problem

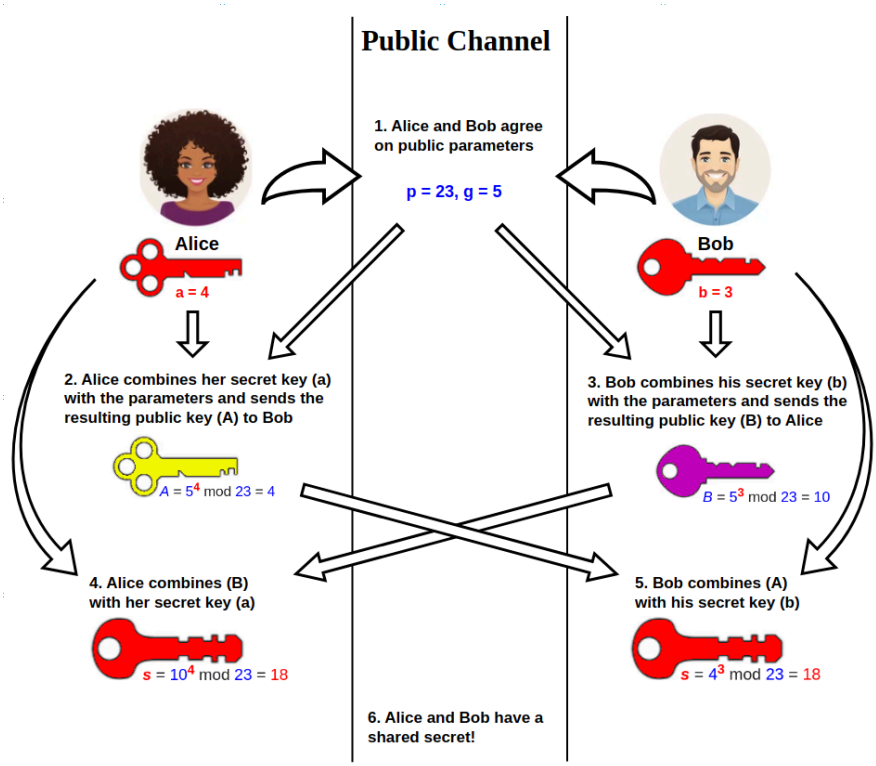

Figure 1: Diffie Hellman key exchange
Link 1: https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_key_exchange

- Before Diffie Hellman, required to exchange keys through secure physical means.
- Diffie Hellman allows for unknown parties to establish a shared secret key over insecure channels.
- This first symmetric key encryption scheme was established in 1976
- The first public key encryption algorithm was established in 1997

- Diffie Hellman is still used today to create ephemeral keys (keys that are used once to establish a public-private pair and then discarded for security) because it's cheap. Used for forward secrecy in TLS
- The RSA cryptosystem was then established for asymmetric key exchange

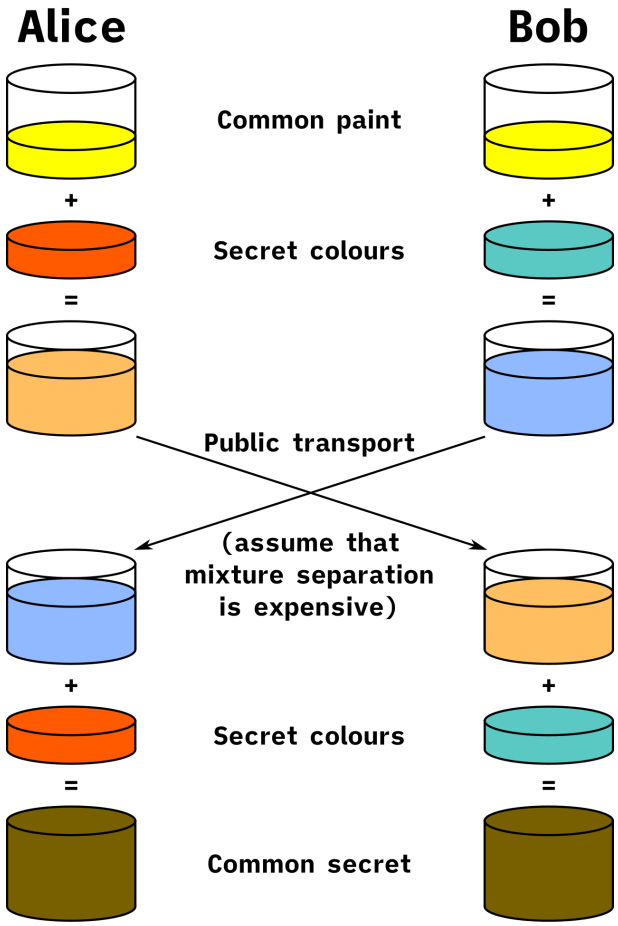

Figure 2: Diffie-Hellman Key Exchange using Paint

- Paint Explanation: Alice and Bob agree on an arbitrary color to start with, then add their own respective secret colors to the mix. They then send these randomly colored mixtures to each other and mix it again with their secret colors, resulting in a common final color

Cryptographic Explanation:

- Both parties choose a prime number as the modulus *p* and integer *g* which is a primitive root module p
- Primitive root is found by determining which root module *p* results in all integers 1, *p*-1. So:

The number 3 is a primitive root modulo 7[5] because

$$\begin{aligned}
3^1 &= 3^0 \times 3 \equiv 1 \times 3 = 3 \equiv 3 \pmod 7 \\
3^2 &= 3^1 \times 3 \equiv 3 \times 3 = 9 \equiv 2 \pmod 7 \\
3^3 &= 3^2 \times 3 \equiv 2 \times 3 = 6 \equiv 6 \pmod 7 \\
3^4 &= 3^3 \times 3 \equiv 6 \times 3 = 18 \equiv 4 \pmod 7 \\
3^5 &= 3^4 \times 3 \equiv 4 \times 3 = 12 \equiv 5 \pmod 7 \\
3^6 &= 3^5 \times 3 \equiv 5 \times 3 = 15 \equiv 1 \pmod 7
\end{aligned}$$

The remainders 3, 2, 6, 4, 5, 1 include each congruence class relatively prime to 7. Higher powers repeat the same pattern periodically.

Figure 3: Determination of how to find that 3 is a primitive root module 7 Link 2: https://en.wikipedia.org/wiki/Primitive_root_modulo_n

1. Alice and Bob publicly agree to use a modulus $p = 23$ and base $g = 5$ (which is a primitive root modulo 23).
2. Alice chooses a secret integer $a = 4$, then sends Bob $A = g^a \bmod p$
   - $A = 5^4 \bmod 23 = 4$ (in this example both A and a have the same value 4, but this is usually not the case)
3. Bob chooses a secret integer $b = 3$, then sends Alice $B = g^b \bmod p$
   - $B = 5^3 \bmod 23 = 10$
4. Alice computes $s = B^a \bmod p$
   - $s = 10^4 \bmod 23 = 18$
5. Bob computes $s = A^b \bmod p$
   - $s = 4^3 \bmod 23 = 18$
6. Alice and Bob now share a secret (the number 18).

Both Alice and Bob have arrived at the same values because under mod p,

$$A^b \bmod p = g^{ab} \bmod p = g^{ba} \bmod p = B^a \bmod p$$

More specifically,

$$(g^a \bmod p)^b \bmod p = (g^b \bmod p)^a \bmod p$$

Figure 4: Explanation of how the resulting example results in Alice and Bob reaching the same shared secret and the math for why this function works. Back to Link 1

Only 42 minutes left after completing this section

Symmetric Key Encryption

Link 3: https://www.geeksforgeeks.org/computer-networks/symmetric-key-cryptography/

- In this form of encryption, the same keys are used for data encryption and decryption. This is used for data security since the same key can be used for both use cases.
- One of the first is the caesar cipher, and most primitive symmetric key encryption used simple substitution techniques (replacing letters and characters in the plaintext to create the ciphertex)

- Plaintext: Message without encryption
- Ciphertext: Encrypted Message
- The eventual culmination of this substitution technique is the one time pad (OTP) which is a theoretically impossible cipher where the key is a random string of characters that is exactly as long as the message itself. The key is then used for a single encryption and then discarded. Downsides: Key must be the same length of the message which still leaks (exposes) information about the message itself. Not practical for larger messages

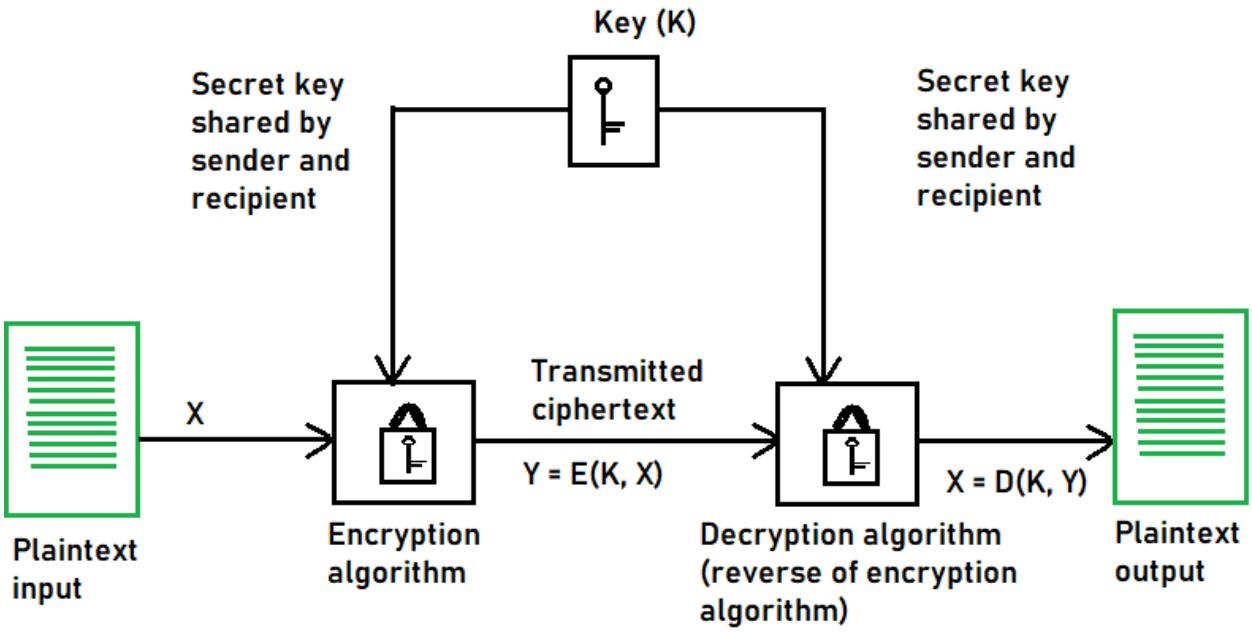

Figure 5: Diagram of Symmetric Key Encryption

- Transposition Techniques rearrange the order of elements in the plaintext without changing the elements themselves. Also not commonly used.
- Most symmetric key cryptography is done with stream ciphers and block ciphers.
- Stream Ciphers:
    - Generate a pseudo-random keystream made up of the encryption key and the unique randomly generated number known as the nonce. The result is a random stream of bits corresponding to the length of the ordinary plaintext.
    - Pseudorandom: (my own definition from prior experience) As close to randomness that we can possibly get. Assumed that pseudorandom generators are created in all further explanations
    - The plaintext is also deciphered into single bits and those bits are joined one by one to the keystream bits by using the XOR bitwise operations. This creates the ciphertext.
    - The recipient then must generate a new keystream made during the encryption to decrypt the ciphertext.
    - The most common stream cipher algorithms are (directly copied from the article):

    - Rivest Cipher 4 (RC4)
        - Strengths: The initial appeal of RC4 came from its efficient design and capability to handle variable-length data streams.
        - Current Status: Due to these identified weaknesses, RC4 is no longer considered secure for most applications. Its use is strongly prohobited by cryptographic standards bodies.
    - Salsa20
        - Strengths: It's fast and efficient, with a simple and elegant design. Most importantly, the security it offers against known attacks is robust. Apart from that, Salsa20 serves as a building block for other cryptographic protocols, exhibiting its versatility.
        - Current Status: Salsa20 is a very widely used and well-respected stream cipher. It's used for many applications where performance and security balance.
    - Grain-128
        - Strengths of Grain-128 include efficiency, lightweight implementation, and the ability to perform well with limited processing power and memory, making it ideal for radio frequency identification (RFID) tags and sensor networks. Importantly, Grain-128 still provides strong security with such simplicity.
        - Current Status: Grain-128 is useful in some resource-limited situations where an application needs to be run with huge restrictions in the amount of data that is available for use.
- Block Cipher
    - The block cipher uses a sequence of blocks that are then encrypted with the key. The output is a sequence of blocks of encrypted data. The receiver then uses the same cryptographic key to decrypt the ciphertext blockchain to the plaintext message.
    - The most common block cipher algorithms are
    - Advanced Encryption Standard (AES)
        - It has support for three-length keys: 128 bits, 192 bits, or 256 bits, the most commonly used one is a 128-bit key.
        - It includes secure communication, data encryption in storage devices, digital rights management (DRM), and so on.
    - Data Encryption Standard (DES)
        - In DES, the 64-bit blocks of plaintext are encrypted using a 56-bit key.
        - This weakness caused by the small key size led to the development of a more secure algorithm, called AES.
    - Triple Data Encryption Algorithm (Triple DES)
        - The development of the Triple DES, also called Triple-DES or TDEA, was triggered by the weak security resulting from the small key size in the DES.
        - Triple DES denotes a method of three times applying the DES algorithm sequentially (encrypt-decrypt-encrypt) on every plaintext block.

- Symmetric Key Cryptography is used in:
    - Data encrypting/decrypting: Protexts sensitive data statically stored in some device
    - Secure Communication: SSL/TLS uses it commonly
    - Authenticity verification: Used by being applied to message authentication codes (MACs) and keyed-hash MACs (HMACs) to authenticate the messages by verifying their authenticity and integrity, thus ensuring tamper-resistant communication.
    - File and disk encryption: used to encrypt sensitive data stored in hard disks or portable storage devices
    - Virtual private Networks: VPNs sometimes use symmetric or asymmetric key encryption to connect remote users and corporate networks.

Didn't get a chance to write whole part in my own words, copied from text:

- Advantages of Symmetric Key Cryptography
    - Speed and efficiency: Symmetric key+ algorithms are better suited for encrypting large volumes of data or for use in real-time communication scenarios as they are faster and less resource-intensive than asymmetric cryptography. SKC algorithms do not involve algebraically mathematical operations.
    - Scalability: Because symmetric key algorithms have relatively low computational overhead, they scale well with the number of users and the amount of data being encrypted.
    - Simplicity: Symmetric encryption protocols are often more straightforward to implement and understand than asymmetric key methods, and this would go a long way in attracting developers and users.
- Challenges and Limitations of Symmetric Key Cryptography
    - Key management and distribution: Both the sender and the receiver in the SKC of a message need to have the same key, and the key should not be seen by a third party. In case the key is somehow captured or compromised by a third party, the security of the encrypted data is also lost.
    - Non-repudiation: Non-repudiation refers to the ability to prove that a specific party has sent a message. In SKC, since the same key is used for encryption and decryption, it is impossible to find out which party created a particular cipher text.
    - Because of these challenges, symmetric key cryptography is often used in conjunction with asymmetric key cryptography.

Only 23 minutes left

Public Key Cryptography (Asymmetric Key Cryptography)

Link 4: https://www.cloudflare.com/learning/ssl/how-does-public-key-encryption-work/

- A method of encrypting data with two different keys and making one of the keys, the public key, available for anyone to use.
- Private Key: The other key that is kept by each party to themselves
- Data encrypted with the public key can only be decrypted with the private key.

- Public key cryptography is also known as asymmetric cryptography.
- Used widely
- Most common:
    - TLS/SSL
    - HTTPS (and so the internets security) because of asymmetric key cryptography
- Cryptography keys are a piece of information used for scrambling data so that it appears random
- TLS/SSL use public key cryptography for establishing secure communications over the internet via HTTPS.
- Websites SSL/TLS certificate contains the public key and the private key is owned by the website
- TLS handshakes use public key cryptography to authenticate the identity of the private key, and to exchange data that is used for generating the session keys (which are generated with symmetric key encryption).
- A key exchange algorithm uses the public-private key pair to agree upon session keys once the handshake is complete.
- Each time client and server reconnect, a new session key is generated so bad actors can't use previously known session keys to hijack the connection between the client and server.

Tuesday 5/5 Notes

Summary:

The key exchange problem aims to solve how two unknown or known parties can talk over the internet without having their message discovered. One of the founding asymmetric encryption schemes to do so is the Diffie-Hellman problem. This problem can be explained in two ways, the mixing of paint, and the cryptographic way. The paint example is explained like this. Say two parties connect to each other and choose an arbitrary color of paint to start with for their message. They use that paint and mix it with their own secret paint color. They then send the mixed paint to each other and mix it again with their secret paint color. The final resulting paint color is the same for both parties. The cryptographic explanation of this is using a prime number as a modulus, finding a root for that prime that finds all integers p-1 when done to the exponent of 0 to p-1. This number is then used as the shared key. So an agreed prime p and root g are used. A secret integer from 0 to p-1 is made by both parties and used to encrypt the message. The messages are sent to each other, then the secret is used again on the message from the other party to form the same secret for both parties.

Symmetric key encryption is the original form of encryption that uses a known secret between two parties and has two techniques, substitution and transformation. Substitution was used for most of history, changing words and phrases by substituting different letters and symbols. The key was used for encryption and decryption of the data in this case. Symmetric key encryption is still used today standalone when data is locally encrypted and can also be decrypted locally. This includes hardware encryption, message authentication codes (used to verify that the user

is who they say they are), and some local software encryption. The problem with symmetric key encryption is the need to share the secret between the two parties. This is mitigated by using asymmetric key encryption to establish a connection and shared secret between two parties, then using the shared secret to either establish a new shared secret and begin using symmetric key encryption, or just using that shared secret to begin exchanging messages using symmetric key encryption. The main algorithms used now are block ciphers.

Asymmetric Key Encryption or Public Key encryption uses two keys instead of one, a public key, and a private key, to exchange messages between two parties. This is the most widely used version, being used by TLS, SSH, and the only way we were able to establish HTTPS. (At this point I didn't check the time and realized it's been 13 minutes, but I'll continue what I was writing since I don't want to stop. Just know at this point, I am over time). Public Key Encryption shares the public key between the two parties and encrypts the messages with the secret key to establish a shared secret between the two parties. The advantage of this is that a shared secret does not need to be established between two parties, unknown parties can interact with each other, and no plaintext data needs to be sent through the web. The disadvantages are the size and complexity constraints. So, this is used in tandem with symmetric key encryption after the session is established for quick communication between the two parties.

Felt Fuzzy or Unclear:

- Disadvantages of public key cryptography
- How block ciphers work
- Main use cases of standalone symmetric key encryption
- Advantages of standalone symmetric key encryption
- For public key cryptography, if the shared secret is what is then used for symmetric key encryption or if a new key is generated
- What part of the shared secret is the message and what part is the actual secret key.
- How do you decipher that ciphertext

Main Focus:

- Asymmetric Key Decryption Explanation
- TLS Session Keys
- Can spend time on:
    - Conceptual understanding of block cipher encryption and decryption
    - Further clarification on public key cryptography (this will also be explored more in depth next week)

TLS Session Keys

Link 5: https://www.cloudflare.com/learning/ssl/what-is-a-session-key/

- Session key is a symmetric cryptographic key used to encrypt one session only.
- Used to encrypt and decrypt data sent between two parties
- Reset every time a new session is established

- Session keys are generated during the TLS handshake
- A session takes place over a network and begins when two devices acknowledge each other to open a virtual connection
- Ends when a close_notify message is sent by both parties or due to inactivity
- HTTPS which is HTTP in combination with the TLS protocol uses both symmetric and asymmetric cryptography in the communication between two parties.
- Asymmetric cryptography is crucial for making the TLS handshake work.
- During the TLS handshake, the client, which is typically a user device like a laptop or smartphone, and a server, which is any web server that hosts a website also:
    - Negotiate the symmetric cryptographic algorithm to use during the TLS handshake
    - Authenticate the server's identity against its TLS certificate also during the asymmetric cryptography process.
- A master secret is generated in a TLS handshake using some random data sent by the client, random data sent by the server, and random data from what's called a "premaster secret" to generate this master secret
- The master secret is used to calculate several session keys for use during that session

Link 6: https://en.wikipedia.org/wiki/Transport_Layer_Security

- Used as further clarification and make sure I solve the first question about how session keys are created
- Clients that want to communicate via HTTPS typically communicate over port 443 or make specific STARTTLS requests to the server to use the TLS connection because HTTP or unencrypted traffic is the norm if not specified.
- After TLS is agreed upon, they negotiate a stateful or remembered connection between the client and server, by using a TLS handshake.
- Asymmetric ciphers are used to establish not only cipher settings but also session-specific shared keys with which further communications is encrypted using a symmetric cipher.
- The TLS handshake works like this:
    - Client initiates connection to a TLS-enabled server and sends a list of supported ciphersuites (or list of algorithms the client supports) to the server
    - Server picks a cipher and hash function that it also supports and notifies the client of the decision
    - The server also then provides the client identification via a digital certificate (won't expand)
    - Client confirms validity of the certificate before proceeding.
    - To generate session keys used for the secure connection the client either:
        - Encrypts a random PreMasterSecret with the server's public key and sends the result to the server (which then can only be decrypted by the server's private key). Both parties then use the random number to generate a unique session key for subsequent encryption and decryption of data during the session

    - Use Diffie-Hellman key exchange or its variant Elliptic-Curve DH to generate a random and unique session key for encryption and decryption that has the additional property of forward secrecy.
    - TLS 1.3 only allows for the forward secrecy approach while LTS 1.2 and below typically use the PreMasterSecret approach

I realized at this point that I only had 10 minutes left to understand public key cryptography decryption so I stopped reading the article

Asymmetric Key Decryption Explanation

Link 7: https://www.ibm.com/think/topics/asymmetric-encryption

- Asymmetric encryption is encryption that uses a public and private key to encrypt and decrypt data
- Generally regarded as more secure, but less efficient than symmetric encryption
- Encryption definition: Process of transforming readable plaintext into unreadable ciphertext to mask sensitive information from unauthorized users
- Asymmetric encryption allows for anyone to use the public key to encrypt data, however, only the holders of the corresponding private key can decrypt that data
- Advantage: Eliminates the need for a secure key exchange which is the main disadvantage of symmetric encryption
- Disadvantage: Notably slower and more resource intensive than symmetric encryption.
- Most organizations rely on a hybrid encryption method that uses asymmetric encryption for secure key distribution and symmetric encryption for subsequent data exchanges.
- Organizations typically choose symmetric encryption when speed and efficiency are crucial or when dealing with large volumes of data over a closed system, such as in a private network.
- Asymmetric encryption is used when security is paramount, such as encrypting sensitive data or securing communication with an open system
- Asymmetric encryption also enables the use of digital signatures, which verify the authenticity and integrity of message to ensure it hasn't been tampered with during the transmission
- AES is the main one with key lengths 128, 196, and 256 bits by most organizations and governments worldwide
- Generally, the longer the key size the higher the security.
- Asymmetric encryption is known for having much longer key lengths than symmetric encryption
- The public key encrypts data or verifies digital signatures and can be freely distributed and shared.
- The private key decrypts data and creates digital signatures but must stay secret to ensure security (end here. Ran out of time)

Wednesday 5/6 Notes

- Still using Link 7. Continuing notes
- The sender encrypts a message using the other party's public key, and the other party uses their private key to decrypt the information.
- Comparison to a locked mailbox. Anyone can put mail in, but only the owner of the mailbox can unlock it to see what's inside
- Asymmetric encryption can also be used to ensure authentication by using the sender's private key to encrypt the message and the other party can use the sender's public key to decrypt and verify it was the original sender who sent it.
- Asymmetric encryption is typically implemented through a public key infrastructure which is a framework for creating, distributing and validating pairs of public and private keys.
- For a hybrid configuration example:
    - Alice generates a pair of public and private keys she shares her public key with Bob
    - Bob generates a symmetric key
    - Bob uses Alice's public key to encrypt the symmetric key, and then he sends the encrypted key to Alice. A threat actor can only intercept the encrypted key without being able to decrypt it
    - Alice receives the encrypted key and uses her private key to decrypt it. Now, Alice and Bob have a shared symmetric key.
- Most common asymmetric encryption algorithms:
    - Rivest-Shamir-Adleman (RSA)
    - Elliptic Curve Cryptography (ECC)
    - Digital Signature Algorithm (DSA)
- Asymmetric encryption use cases:
    - Web Browsing
    - Secure communications
    - Digital Signatures
    - Authentication
    - Key Exchange
    - Blockchain technology

IBM Notes Summary:

Public key or asymmetric cryptography uses two keys: a publicly shared key that anyone can use for encrypting messages, and a private key that is kept by the generator for decrypting messages. The most common algorithms are RSA, ECC, and DSA and it's mainly used in web browsing, secure communications, digital signatures, authentication, key exchange, and blockchain technology. A great acronym to explain public key cryptography is imagine a locked mailbox with a slit to put mail in and a collection box at the bottom. Anyone can put mail in by using that slit (public key) and now no one knows what's in that letter (ciphertext), but only the one with the key can unlock the collection box (secret key) and read the letters (plaintext). In

cryptography terms, one party will generate a public private key and send the public key to the other party. The other party will then use that public key to encrypt their message and send the ciphertext to the main party. That party will then use their private key to decrypt the ciphertext and read the message. This can then be implemented to share session keys in TLS through two approaches. 1) From that initial example, instead of encrypting a message, the other party encrypts their own symmetric key and sends that over as ciphertext to the main party. When the main party decrypts that ciphertext, they'll now have the session key to start sending messages to one another. 2) Use the Diffie Hellman Key Exchange to establish a shared secret session key to continue exchanging messages between each party. These both are the hybrid approach using both asymmetric encryption first to establish the connection and symmetric encryption to exchange messages as the encryption which keeps people and companies messages and secrets safe from threat actors. **Didn't mention: The authentication piece using the private key to encrypt so the other party can verify with the public key**

Overall Summary:

What is encryption, why do we need it, what problem did symmetric encryption leave unsolved, and how do asymmetric and symmetric encryption work together to keep the internet secure today?

Encryption is the method used to keep messages and secrets safe from threat actors in the digital space. We need encryption to keep banking information, text messages, facetime videos, company secrets, etc. from being leaked and used by bad actors to extort our time, money, and emotions from their own gains. Encryption prevents all this by keeping all these types of interactions within the digital space safe. The first instance of cryptography was symmetric encryption which kept messages private and safe as long as both parties could physically meet and exchange a shared key to encrypt and decrypt messages. The problem with the digital space is that it's impossible to share a key with a user in another country physically to encrypt and decrypt messages. So, asymmetric or public key encryption was created to alleviate this issue. Asymmetric key encryption uses two keys, a public key that can be shared with anyone and everyone for encrypting messages, and a private key that is not shared to decrypt messages. Asymmetric encryption can be explained using a locked mailbox analogy with a slit to put letters in and a locked collection box at the bottom. Anyone can put their letters (plaintext) into the slit (public key). Once it's in the collectors box, no one can read that message anymore (ciphertext). At a later time, the owner of the box can bring their key to the collection box (secret key), open the collection box, and read the messages (plaintext). The downside to asymmetric encryption is its large keysize leading to more computation resources to only use asymmetric key encryption for large amounts of data. So, a hybrid approach was invented using asymmetric key encryption to create a shared symmetric key. If we go back to our analogy, this is done by rather than writing a letter, the person who wants to establish a connection with the owner of the locked mailbox puts their address (symmetric key) into the slit for further exchange between the two people. In cryptography terms this is done by having two parties Alice and Bob. Alice generates a public and private key (with asymmetric encryption algorithms) and sends the public key to Bob. Bob generates a session key (using symmetric encryption algorithms) and

encrypts it with Alice's public key. Bob then sends it to Alice. She decrypts the message using her private key. Now they have the same shared key and can send messages to each other. This keeps the internet secure by using this basic structure for protocols used in web browsers, authentication and digital signatures, secure communication software (WhatsApp and Signal), etc. **Critics: The history is slightly off. Symmetric encryption didn't require physical meetings as its *defining limitation* — it was used fine for centuries with physical key exchange. The real problem was specifically the digital space making physical exchange impossible at scale. You actually say this correctly two sentences later, so just trim the first mention to avoid the slight contradiction. Diffie-Hellman disappeared. Your week started with DH as the founding solution to the key exchange problem, but this narrative skips straight from "problem" to "asymmetric encryption." One sentence acknowledging that DH was the first cryptographic solution to key exchange over an insecure channel would complete the arc. It also sets up Week 3 when DH comes back in the TLS handshake.**

Thursday 5/7 Tying it all Together

Nothing to report here

Week 1 Retrospective

AI Generated:

What you did well

- You showed up every single day. After a month and a half of disruption, that's the hardest part and you did it. No missed days.
- Your recall was genuinely strong. That cold summary you wrote Wednesday night from pure memory was accurate, organized, and in your own voice. That's not common for someone who felt like they were struggling all week.
- You course-corrected in real time. You identified your own mistakes — the transcription problem, the rabbit hole on stream ciphers, losing focus on the weekly goal — without me having to point them out first. That self-awareness is more valuable than any single piece of content knowledge.
- The mailbox analogy is legitimately yours and it's good. You didn't borrow it, you developed it organically and then extended it creatively to explain the hybrid approach. That's original thinking.
- You finished. A complete presentation exists. That directly attacks the decision paralysis and overplanning pattern you named in week one.

What was bad

- You transcribed instead of processed for most of Monday and parts of Tuesday. When fatigue hit you defaulted to copying text rather than stopping, stepping back, and asking what you actually needed to understand. Copying feels productive but it's the least effective thing you can do with limited time.
- You lost your goal anchor almost immediately. By the middle of Monday's session you were deep in stream cipher algorithms that had nothing to do with explaining encryption to your family. You didn't have a compass sentence visible and it cost you significant time and energy every single day.
- The research hours were inconsistent in quality. Monday's last 20 minutes and Tuesday's asymmetric section were noticeably weaker than your best work. You pushed through fatigue instead of recognizing when diminishing returns had set in and stopping cleanly.
- You compressed Tuesday and Wednesday into one day. I understand why and the reasons were legitimate, but the consequence was real — you had no true consolidation day. Wednesday was supposed to be the day you stopped learning new things and started connecting everything. That never happened cleanly, and you felt it on Thursday when you had to build the presentation without a fully settled understanding of the material.
- The presentation has no narration bridge between slides. Each slide is an island. A family audience following along will have moments of "wait, why are we looking at a mailbox now" without spoken transitions connecting the story. This is partly a time constraint problem but also a structural habit to build — every slide needs a one sentence spoken bridge from the previous one.

What to fix next week specifically

- Write your goal sentence before you start Monday's research and keep it visible the entire hour. Not in your notes — physically visible on a sticky note or second window. Check every rabbit hole against it.
- When you catch yourself transcribing, stop completely. Close the article. Write what you just read in one sentence from memory. If you can't, read it again. If you can, move on. Never copy paste or directly transcribe again.
- Do the talk-through on Thursday before you build slides, not after. Ten minutes of speaking the narrative out loud will show you the gaps in your story faster than re-reading notes will.
- Protect Wednesday as a genuine consolidation day. No new sources. Just your own notes and your own voice connecting the pieces.

Bottom line

- You did enough to call this week a success by the only metric that actually matters — you finished and you're presenting. But you left real performance on the table through discipline issues that are entirely fixable. Next week you know what they are. Now fix them.

Friday 5/8 Remarks from Presentation

- The first slide should not just include the hook, but explain what encryption is first, then give the hook
- Make sure that the presentation includes the 5 Ws and How
- The definition of encryption should not include the work to encrypt in it. Better to use the phrase “scramble the message” instead of “encrypt the message”.
- Add to my slide deck
    - Objective
    - Plan (Agenda)
- Should review the slides before giving the presentation
- Need to improve presentation skills (will come with more time doing the presentations). To be honest, I want to have these improvements come more naturally and not have to rely on a script when giving a presentation. I expect as I do this more often, I will get better and better at giving my presentations
    - Reading off the slides too much on the reading slide
    - Stuttering and repeating words when trying to go over a narrative for each slide
    - Need better transitions between slides
    - (Personal Remarks for presentations skills):
        - Needed to remember what forward secrecy was so I could better present that transition and the need for the Diffie-Hellman key exchange over RSA for TLS 1.3
        - Overall need to stay grounded in following a narrative
        - Spend more time looking at my audience then looking at my slides.
- Before beginning to research a topic, spend a week picking a topic and determining all material needed to learn that topic and how to split it up into weekly learning presentation topics and objectives. For the presentation that week, go over an agenda of what will be covered each week, overall objectives, and weekly objectives. This keeps me grounded during the presentation and during each week's research. Be open to feedback on if I think I need to pivot during a week, or if during my research process, I find something else that needs to be researched to continue learning that topic if it needs its own week of learning to do so.

I think that this could definitely be fixed if I allow myself 5 minutes (only) to read through my slides on Friday before I give my presentation and allow myself time to fix the slides if I deem fit. My mom also suggested that I fix these slides up now, so that I can combine them after I finish this whole Public Key Infrastructure lesson and make it into one slide deck for all my presentations over this week. I can then give this 4 or 5 week (I’m not sure) presentation to the people at work. So, I need to work on developing a more thorough roadmap for how long I

should focus on Public Key Infrastructure, come up with retroactively the objectives I wanted to accomplish this week, and figure out the objectives for the weeks to come.